

Core Source Code Analysis of Embodied Intelligence Features

Course Content

1. The AI large model embodied intelligence feature is a composite function involving multiple node programs working together. This section explains the core programs in detail.

Source Code Package Structure

Package File Structure

The AI large model embodied intelligence ros package is called "largemodel", and the package path is:

```
/home/yahboom/LargeModel_ws/src/largemodel/largemodel
```

The functions of folders and files are described as follows:

config

Configuration folder, storing configuration files

- large_model_interface.yaml
Large model interface configuration file, used to configure API-KEY for various platforms and large model parameters
- yahboom.yaml
Node configuration file: used to configure parameters for core nodes

largemodel

Source code folder, core programs for AI large model embodied intelligence features

- asr.py
Speech recognition program file, used to run speech recognition models for user interaction
- model_service.py
Model server program file, used to call various model interfaces to implement model inference architecture
- action_service.py
Action server program file, used to receive action lists from the model server, control robot movement and play sounds

launch

Launch files folder, used to store ros2 node launch files

- largemodel_control.launch.py
Launch file for AI large model embodied intelligence features: launches multiple nodes, with two launch modes: voice interaction mode and text interaction mode

resources_file

Stores system audio files

utils

Utilities folder, storing non-core function program files

- large_model_interface.py

Large model interface program file, contains underlying interfaces for calling large models on various platforms. Different platforms and models have different calling programs, managed uniformly in the interface file

- mic_serial.py

Voice module driver program file, used to drive the wake word function of the voice module

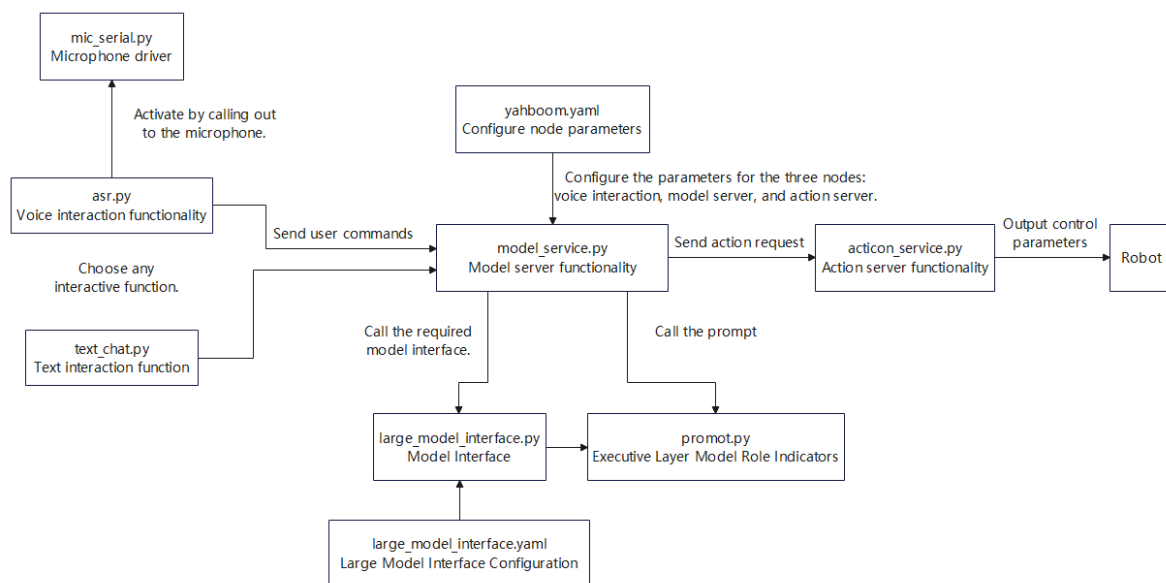
- promot.py

Large model prompt file: used to generate prompts for the execution layer large model

- dify_client2.py

dify API function, used for international version dify to request local dify applications

Program Interrelation Diagram



Voice Recognition Feature

The voice recognition feature includes two parts: VAD (Voice Activity Detection) and speech-to-text conversion. Source code path:

```
/home/yahboom/LargeModel_ws/src/largemodel/largemodel/asr.py
```

VAD Voice Activity Detection

Implementation: The `listen_for_speech` method in the `ASRNode` class

Program Analysis: Records audio in real-time through a specified microphone, uses VAD (Voice Activity Detection) to determine if someone is speaking. When a speech segment ends (continuous silence exceeds the set number of frames), recording stops and valid speech content is saved to a file.

Detailed logic is as follows:

1. Initialize audio stream and configure parameters (such as sample rate, number of channels, etc.).
2. Continuously read audio frames and perform voice activity detection.
3. If speech is detected, add audio frames to the buffer; if continuous silence exceeds the threshold (90 frames, approximately 1s), end recording.
4. After recording ends, remove trailing silence and save valid speech as a WAV file.
5. If no valid speech is detected, do not save the file.

```
def listen_for_speech(self, mic_index=0):
    p = pyaudio.PyAudio()
    audio_buffer = []
    silence_counter = 0
    MAX_SILENCE_FRAMES = 90 # 30 frames * 30ms = 900ms silence to stop
    recording / Stop after 900ms of silence (30 frames * 30ms)
    speaking = False # Voice activity flag / Flag indicating speech
    activity
    frame_counter = 0 # Counter / Frame counter
    stream_kwargs = {
        "format": pyaudio.paInt16,
        "channels": 1,
        "rate": self.sample_rate,
        "input": True,
        "frames_per_buffer": self.frame_bytes,
    }
    if mic_index != 0:
        stream_kwargs["input_device_index"] = mic_index

    # Prompt the user to speak via the buzzer / Prompt the user to speak via
    the buzzer
    self.pub_beep.publish(UInt16(data=1))
    time.sleep(0.5)
    self.pub_beep.publish(UInt16(data=0))

    try:
        # Open audio stream / Open audio stream
        stream = p.open(**stream_kwargs)
        while True:
            if self.stop_event.is_set():
                return False

            frame = stream.read(
                self.frame_bytes, exception_on_overflow=False
            ) # Read audio data / Read audio data
            is_speech = self.vad.is_speech(
                frame, self.sample_rate
            ) # VAD detection / VAD detection

            if is_speech:
                # Detected speech activity / Detected speech activity
                speaking = True
                audio_buffer.append(frame)
                silence_counter = 0
            else:
                if speaking:
```

```

        # Detect silence after speech activity / Detect silence
        after speech activity
            silence_counter += 1
            audio_buffer.append(
                frame
            ) # Continue recording buffer / Continue recording
        buffer

        # End recording when silence duration meets the
        threshold / End recording when silence duration meets the threshold
        if silence_counter >= MAX_SILENCE_FRAMES:
            break
        frame_counter += 1
        if frame_counter % 2 == 0:
            self.get_logger().info("1" if is_speech else "-")
            # print('1-' if is_speech else '0-', end='', flush=True) #
            Real-time status display / Real-time status display
        finally:
            stream.stop_stream()
            stream.close()
            p.terminate()

        # Save valid recording (remove trailing silence) / Save valid recording
        (remove trailing silence)
        if speaking and len(audio_buffer) > 0:
            # Trim the last silent part / Trim the last silent part
            clean_buffer = (
                audio_buffer[:-MAX_SILENCE_FRAMES]
                if len(audio_buffer) > MAX_SILENCE_FRAMES
                else audio_buffer
            )

            with wave.open(self.user_speechdir, "wb") as wf:
                wf.setnchannels(1)
                wf.setsampwidth(p.get_sample_size(pyaudio.paInt16))
                wf.setframerate(self.sample_rate)
                wf.writeframes(b"".join(clean_buffer))
            return True

```

ASR Speech Recognition

Implementation: The ASR_conversion method in the ASRNode class converts recorded audio files to text

Calls the speech recognition model interface function in the large_model_interface.py file

Note: When the recognized text is less than 4 characters, it is considered invalid. This is to prevent content after accidental wake-up from being treated as valid content.**

Program Analysis:

1. If using online_asr, call the corresponding method and check if the result is valid text (length greater than 4). Return recognized content on success, otherwise log the error and return 'error'.
2. Otherwise, use SenseVoiceSmall_ASR for recognition, and perform the same result validation and processing.

Among them, SenseVoiceSmall_ASR is a local model speech recognition method, and can only be used with Jetson Orin Nano and Jetson Orin NX hosts.

```
def ASR_conversion(self, input_file):
    if self.use_online_asr:
        result=self.modelinterface.online_asr(input_file)
        if result[0] == 'ok' and len(result[1]) > 4:
            return result[1]
        else:
            self.get_logger().error(f'ASR Error:{result[1]}')
            return 'error'
    else:
        result=self.modelinterface.SenseVoiceSmall_ASR(input_file)
        if result[0] == 'ok' and len(result[1]) > 4:
            return result[1]
        else:
            self.get_logger().error(f'ASR Error:{result[1]}')
            return 'error'
```

Model Server Feature

The implementation program is model_service.py, used to receive speech recognition results in voice interaction mode or terminal input in text interaction mode, implementing large model inference logic. Source code path:

```
/home/yahboom/LargeModel_ws/src/largemodel/largemodel/model_service.py
```

Dual Model Inference (Domestic Version)

The default uses dual model inference mode. Implementation programs are the dual_large_model_mode and instruction_process methods in the LargeModelService class

Program Analysis:

1. If it is a new instruction cycle (self.new_order_cycle is True):
 - Initialize multimodal history context;
 - Use the task decision model to perform task planning on the user input prompt;
 - Pass the planning results to the instruction execution layer model for processing;
 - Set new_order_cycle to False, indicating the current instruction cycle has begun execution.
2. Otherwise (not a new cycle):
 - Directly pass the input prompt and type to the instruction execution layer model for processing.

```
def dual_large_model_mode(self, type, prompt=""):
    """
    This function implements the dual model inference mode, where the text
    generation model first plans the task, and then the multimodal large model
    generates the action list.
    This function implements the dual model inference mode, where the text
    generation model first plans the task, and then the multimodal large model
    generates the action list.
    """
    if (
```

```

        self.new_order_cycle
    ): # Determine if it is a new task cycle / Determine if it is a new task
cycle
        # Determine if the previous round of dialogue instructions are
completed. If completed, clear the historical context and start a new context /
Determine if the previous round of dialogue instructions are completed. If
completed, clear the historical context and start a new context
        self.model_client.init_Multimodal_history(
            get_prompt()
        ) # Initialize execution layer context history / Initialize execution
layer context history
        execute_instructions = self.model_client.TaskDecision(
            prompt
        ) # Call the decision layer large model for task planning / Call the
decision layer large model for task planning

        if not execute_instructions == "error":
            if self.text_chat_mode:
                msg = String(
                    data=f"The upcoming task to be carried out:
{execute_instructions}"
                )
                self.text_pub.publish(msg)
            else:
                self.get_logger().info(
                    f"The upcoming task to be carried out:
{execute_instructions}"
                ) # Upcoming task to execute:... / Upcoming task to execute:...
                self.instruction_process(
                    type="text",
                    prompt=f"User: {prompt},Decision layer AI planning:
{execute_instructions}",
                ) # Pass the planned execution steps from the decision layer model
to the execution layer model / Pass the planned execution steps from the decision
layer model to the execution layer model
                self.new_order_cycle = (
                    False # Reset the instruction cycle flag / Reset the
instruction cycle flag
                )
            else:
                self.get_logger().info(
                    "The model service is abnormal. Check the large model account or
configuration options"
                ) # Model inference failed, please check if the model quota and
account are normal!!! / Model inference failed, please check if the model quota
and account are normal!!!
        else:
            self.instruction_process(
                prompt, type
            ) # Call the execution layer large model to generate an action list and
execute / Call the execution layer large model to generate an action list and
execute

def instruction_process(self, prompt, type, conversation_id=None):
    """
        Based on the type of input information (text/image), construct different
request bodies for inference and return the result.

```

```

Based on the type of input information (text/image), construct different
request bodies for inference and return the result.
"""
if self.regional_setting == "China": # Domestic version
    if type == "text":
        raw_content = self.model_client.multimodalinfer(prompt)
    elif type == "image":
        self.save_single_image()
        raw_content = self.model_client.multimodalinfer(
            "Robot feedback:seewhat() execution completed",
            image_path=self.image_save_path
        )
        json_str = self.extract_json_content(raw_content)

elif self.regional_setting == "international": # International version
    if type == "text":
        result = self.model_client.TaskExecution(
            input=prompt,
            map_mapping=self.map_mapping,
            language=self.language_dict[self.language],
            conversation_id=conversation_id,
        )
        if result[0]:
            json_str = self.extract_json_content(result[1])
            self.conversation_id = result[2]
        else:
            self.get_logger().info(f"ERROR:{result[1]}")
    elif type == "image":
        self.save_single_image()
        result = self.model_client.TaskExecution(
            input="Robot feedback:seewhat() execution completed",
            map_mapping=self.map_mapping,
            language=self.language_dict[self.language],
            image_path=self.image_save_path,
            conversation_id=conversation_id,
        )
        if result[0]:
            json_str = self.extract_json_content(result[1])
            self.conversation_id = result[2]
        else:
            self.get_logger().info(f"ERROR:{result[1]}")

    if json_str is not None:
        # Parse JSON string, separate "action" and "response" fields / Parse
        JSON string, separate "action" and "response" fields
        action_plan_json = json.loads(json_str)
        action_list = action_plan_json.get("action", [])
        llm_response = action_plan_json.get("response", "")
    else:
        self.get_logger().info(
            f"LargeScaleModel return: {json_str},The format was unexpected. The
            output format of the AI model at the execution layer did not meet the
            requirements"
        )
        return

if self.text_chat_mode:

```

```

        msg = String(data=f'"action": {action_list}, "response":
{llm_response}\'')
        self.text_pub.publish(msg)
    else:
        self.get_logger().info(
            f'"action": {action_list}, "response": {llm_response}\'
        )

    self.send_action_service(
        action_list, llm_response
    ) # Asynchronously send action list and response content to ActionServer /
Asynchronously send action list and response content to ActionServer

```

Dual Model Inference (International Version)

The program implementation logic is the same as the domestic version. The difference is that the international version requests a local dify application, which then requests the cloud large model

```

def dual_large_model_international_model(self, type, prompt=""):
    """
    This function is suitable for the international version dual model inference
    mode, using dify as middleware
    """
    if (
        self.new_order_cycle
    ): # Determine if it is a new task cycle / Determine if it is a new task
cycle
        self.conversation_id = None
        result = self.model_client.TaskDecision(prompt)
        if result[0]:
            if self.text_chat_mode: # Text interaction mode / Text interaction
mode
                msg = String(
                    data=f"The upcoming task to be carried out:{result[1]}"
                )
                self.text_pub.publish(msg)
            else: # Voice interaction mode / Voice interaction mode
                self.get_logger().info(
                    f"The upcoming task to be carried out:{result[1]}"
                )

            self.instruction_process(
                type="text", prompt=f"User: {prompt},Decision layer AI planning:
{result[1]}"
            )
            self.new_order_cycle = (
                False # Reset the instruction cycle flag / Reset the
instruction cycle flag
            )
        else:
            self.get_logger().info(f"ERROR: {result[1]}")

    else:
        self.instruction_process(
            prompt, type, conversation_id=self.conversation_id

```

```
) # Call the execution layer large model to generate an action list and  
execute / Call the execution layer large model to generate an action list and  
execute
```

Action Server Feature

The implementation program is `action_service.py`, used to receive action lists from the model server, parse action lists and execute them. Source code path:

```
/home/yahboom/LargeModel_ws/src/largemodel/largemodel/action_service.py
```

Action Function Library

The robot's action functions come from methods in the `CustomActionServer` class, containing various sub-action functions. Here we explain the function that controls the arm to move upward as an example. Other action functions will be explained when they first appear in subsequent sections.

Program Analysis:

1. Convert the input string parameter to floating point number
2. Call the function directly, pass parameters, communicate with the underlying driver board, control six servos
3. Decide whether to publish completion message based on the mode

```
def arm_up(self): # Arm moves upward  
    # Call the function, communicate directly with the underlying driver board,  
    control six servos to extend upward  
    Arm.Arm_serial_servo_write6(90,90,90,90,90,90,1000)  
    # If not in combination mode and not interrupted, publish message that arm  
    upward control is complete  
    if not self.combination_mode and not self.interrupt_flag:  
        self.action_status_pub("arm_up_done")
```

Parse Action List to Control Robot Feature

Parse the action list generated by the large model into functions that control the robot entity, and execute them. Implementation program is the `execute_callback` method in the `CustomActionServer` class.

Program Analysis:

1. Receive the action list sent by the client (string format);
2. If the action list is empty, return success result directly;
3. If there is only one action, parse and execute the corresponding method, abort if failed;
4. If there are multiple actions, enter combination mode and execute sequentially, recording logs and feedback during the process;
5. After all actions are executed, call `stop()` to stop the robot;
6. Finally return the execution success result to the client.

```
def execute_callback(self, goal_handle):  
    feedback_msg = Rot.Feedback()  
    stop = getattr(self, "stop") # Get the stop method, stop robot movement by  
    publishing topic
```

```

actions=goal_handle.request.actions
if not actions: # If action list is empty, return success directly
    if not self.text_chat_mode:
        self.send_Audiorequest(goal_handle.request.llm_response)

    if self.use_double_llm:
        self.action_status_pub('Robot feedback: User reply completed')

    goal_handle.succeed()
    result = Rot.Result()
    result.success = True
    return result
elif len(actions) == 1:# Action list has only one action, normal execution
process
    # Case 2: List has only one element
    action = actions[0]
    if not self.text_chat_mode:
        self.send_Audiorequest(goal_handle.request.llm_response)

    match = re.match(r"(\w+)\((.*)\)", action)
    if not match:
        self.get_logger().warning(f'action_service: {action} is invalid
action,skip execution')
        goal_handle.abort()
        result = Rot.Result()
        result.success = False
        return result
    else:
        action_name, args_str = match.groups()
        args = [arg.strip() for arg in args_str.split(",")] if args_str else
[]

        try:
            if hasattr(self, action_name):
                method = getattr(self, action_name)
                method(*args)
                feedback_msg.status = f'action service execute {action}
succeeded'

                # self.get_logger().info(feedback_msg.status) # Add log
                goal_handle.publish_feedback(feedback_msg)
            else:
                self.get_logger().warning(f'action_service:invald:
{action_name},skip execution')
                self.action_status_pub('Robot feedback: Action function does
not exist, cannot execute')
                goal_handle.abort()
                result = Rot.Result()
                result.success = False
                return result
        except Exception as e:
            self.get_logger().error(f'action_service:execute action:
{action} failed,error-log: {str(e)}')
            goal_handle.abort()
            result = Rot.Result()
            result.success = False
            return result
    else:# If action list has multiple actions, enable combination mode
    if not self.text_chat_mode:
        self.send_Audiorequest(goal_handle.request.llm_response)

```

```

self.combination_mode=True
for action in actions:
    # Use regex to parse action name and parameters
    match = re.match(r"(\w+)\(((.*)\)", action)
    if not match:
        self.get_logger().warning(f'action_service: {action} is invalid
action, skip execution')
        continue

    action_name, args_str = match.groups()
    args = [arg.strip() for arg in args_str.split(",")] if args_str else
[]

    try:

        # Check if action is a method in CustomActionServer class
        if hasattr(self, action_name):
            method = getattr(self, action_name)
            # Dynamically call method and pass parameters
            method(*args)
            # method(*converted_args)
            feedback_msg.status = f'action service execute {action}
succeeded'

            self.get_logger().info(feedback_msg.status) # Add log
            goal_handle.publish_feedback(feedback_msg)
        else:
            self.get_logger().warning(f'action_service:invald:
{action_name}, skip execution')
            except Exception as e:
                self.get_logger().error(f'action_service:execute action:
{action} failed,error-log: {str(e)}')
                goal_handle.abort()
                result = Rot.Result()
                result.success = False
                return result

        self.action_status_pub(f'Robot feedback: Executed {actions} completed')
        self.combination_mode=False# Reset combination mode flag
    stop() # Stop the robot after executing all actions
    # Return success message to client
    goal_handle.succeed()
    result = Rot.Result()
    result.success = True
    return result

```

Interrupt Feature

The robot supports interruption at any stage. Specifically, it can be divided into recording stage interruption, dialogue stage interruption, and action stage interruption. Here we introduce the principles of interruption at each stage.

Recording Stage Interruption

If you misspoke during recording or are not satisfied with the content and need to re-record, you can directly wake up again **during the recording process** to interrupt the previous recording and start speaking a new one.

- The logic implementation is in the **main_loop** method of the **ASRNode** class in the `asr.py` file:
- Each time when waking up, if there is already a wake-up recording thread running, interrupt it through the thread event `stop_event` and wait for it to finish;
- After clearing the stop event flag, start a new recording thread

```
def main_loop(self):
    while rclpy.ok():
        while (
            self.audio_request_queue.qsize() > 1
        ): # Process only the most recent wake-up request to prevent duplicates
            / Process only the most recent wake-up request to prevent duplicates
            self.audio_request_queue.get()

        if not self.audio_request_queue.empty():
            self.audio_request_queue.get()
            self.wakeup_pub.publish(
                Bool(data=True)
            ) # Publish wake-up signal / Publish wake-up signal
            self.get_logger().info("I'm here")
            playsound(
                self.audio_dict[self.first_response]
            ) # Respond to the user / Respond to the user

            if (
                self.current_thread and self.current_thread.is_alive()
            ): # Interrupt the previous wake-up handling thread / Interrupt the
                previous wake-up handling thread
                self.stop_event.set()
                self.current_thread.join() # wait for the current thread to
                finish / wait for the current thread to finish
                self.stop_event.clear() # Clear the event / Clear the event
                self.current_thread = threading.Thread(target=self.kws_handler)
                self.current_thread.daemon = True
                self.current_thread.start()
            rclpy.spin_once(self, timeout_sec=0.1)
            time.sleep(0.1)
```

Dialogue Stage Interruption

If during the robot's speech you feel the robot's reply is unsatisfactory or you don't want the robot to continue replying, you can directly interrupt the robot's speech using the wake word and start speaking a new recording. At this point, you can give the robot a new instruction (still in the current task cycle), or say "end current task" to end the current task and start a new task cycle.

- The logic implementation is in the **wakeup_callback** and **play_audio** methods of the **CustomActionServer** class in the `action_service.py` file:
- `wakeup_callback` is the wake-up handling callback function. Each time the `asr.py` program wakes up, it publishes the wake-up signal through topic communication. `wakeup_callback` subscribes to and processes the signal.

- Each time waking up, check if **pygame.mixer** is currently playing audio. If so, notify the playback thread to stop playing through the thread event `self.stop_event`
- If the previous action is detected to be running after wakeup, set the **self.interrupt_flag** flag for subsequent action interruption

```
def wakeup_callback(self, msg):
    if msg.data:
        if pygame.mixer.music.get_busy():
            self.stop_event.set()
        if self.action_runing:
            self.interrupt_flag = True
            self.stop()
            self.pubSix_Arm(self.init_joints)
```

`play_audio` checks if `self.stop_event` is set when playing audio. Once set is detected, immediately stop the currently playing audio.

```
def play_audio(self, file_path: str, feedback: bool = False) -> None:
    """
    Synchronous audio playback function / The function for playing audio in
    synchronous mode
    """
    pygame.mixer.music.load(file_path)
    pygame.mixer.music.play()
    while pygame.mixer.music.get_busy():
        if self.stop_event.is_set():
            pygame.mixer.music.stop()
            self.stop_event.clear() # Clear event
            return
        pygame.time.Clock().tick(10)
    if feedback:
        self.action_status_pub("Robot feedback: User reply completed")
```

Action Stage Interruption

If the robot is woken up during action execution, it stops the current action and returns to the initial posture.

For example, actions like robotic arm gripping and color block sorting require starting external programs in subprocesses. Here we explain using the color block handling function `change_pose` as an example:

When the color block handling is not completed, it will keep waiting in the loop `while not self.change_pose_future.done():`. If the `self.interrupt_flag` interrupt flag is set during this process, the `check_close_change_pose` function is called first, which recursively ends the child process tree. Then the robotic arm is returned to the gripping posture, and the task is ended.

```
def change_pose(self, x1, y1, x2, y2, x3, y3, x4, y4, src, tar, side):
    src_color = src.strip('"\'') # Remove single and double quotes
    self.cur_down_name = src_color
    self.cur_down_pose[self.cur_down_name] = []
    cmd1 = "ros2 run largemodel_arm Get_Target_Pose_KCF"
    subprocess.Popen(
        [
            "gnome-terminal",
```

```

        "--title=grasp_desktop",
        "--",
        "bash",
        "-c",
        f"{cmd1}; exec bash",
    ]
)
time.sleep(3.0)
x1 = int(x1)
y1 = int(y1)
x2 = int(x2)
y2 = int(y2)
x3 = int(x3)
y3 = int(y3)
x4 = int(x4)
y4 = int(y4)
side = int(side)
time.sleep(5.0)
self.object_position_pub.publish(Int16MultiArray(data=[x1, y1, x2,
y2,x3,y3,x4,y4,side]))

while not self.change_pose_future.done():
    if self.interrupt_flag:
        self.check_close_change_pose()
        Arm.Arm_serial_servo_write6(90,150,12,20,90,30,1000)
        return
    time.sleep(0.1)

```